Practical 13: MiniBank Generic Repository Pattern with Reflection @Id

Problem Definition:

Build a universal, reusable generic Repository<T> backed by a HashMap. Use Java reflection inside save to dynamically discover the field marked with custom @Id annotation and extract its value as the primary key. Implement varargs saveAll and bounded generic methods, proving reusability across Account and Customer entities.

Code:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {}

class Account {
    @Id
    private String accountNumber;
    private String ownerName;
    private double balance;

    public Account(String accountNumber, String ownerName, double balance) {
        this.accountNumber = accountNumber;
        this.ownerName = ownerName;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public double getBalance() {
        return balance;
    }

    @Override
    public String toString() {
        return String.format("Account[No=%s, Owner=%s, Balance=Rs. %.2f]", accountNumber,
ownerName, balance);
    }
}

class Customer {
    @Id
```

```
    private String customerId;
    private String fullName;
    private String email;

    public Customer(String customerId, String fullName, String email) {
        this.customerId = customerId;
        this.fullName = fullName;
        this.email = email;
    }

    @Override
    public String toString() {
        return String.format("Customer[ID=%s, Name=%s, Email=%s]", customerId, fullName, email);
    }
}

class Repository<T> {
    private final Map<String, T> storage = new HashMap<>();

    public T save(T entity) {
        try {
            String id = extractId(entity);
            if (id == null) {
                throw new IllegalArgumentException("Entity lacks an @Id annotated field with
non-null value");
            }
            storage.put(id, entity);
            return entity;
        } catch (IllegalAccessException e) {
            throw new RuntimeException("Reflection access failed: " + e.getMessage(), e);
        }
    }

    @SafeVarargs
    public final void saveAll(T... items) {
        for (T item : items) {
            save(item);
        }
    }

    public T findById(String id) {
        return storage.get(id);
    }

    public ArrayList<T> findAll() {
        return new ArrayList<>(storage.values());
    }

    public boolean delete(String id) {
        return storage.remove(id) != null;
    }

    public int count() {
        return storage.size();
    }

    public static <N extends Number> double average(N[] values) {
        if (values == null || values.length == 0) return 0.0;
```

```
        double sum = 0.0;
        for (N val : values) {
            sum += val.doubleValue();
        }
        return sum / values.length;
    }

    private String extractId(T entity) throws IllegalAccessException {
        Class<?> clazz = entity.getClass();
        while (clazz != null && clazz != Object.class) {
            for (Field field : clazz.getDeclaredFields()) {
                if (field.isAnnotationPresent(Id.class)) {
                    field.setAccessible(true);
                    Object val = field.get(entity);
                    return val != null ? val.toString() : null;
                }
            }
            clazz = clazz.getSuperclass();
        }
        return null;
    }
}

public class PartB {
    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 13 - Part B: MiniBank Generic Repository Pattern with
Reflection @Id");
        System.out.println();

        Repository<Account> accountRepo = new Repository<>();

        Account a1 = new Account("AC1001", "Rudra Modi", 75000.0);
        Account a2 = new Account("AC1002", "Aarav Patel", 42000.0);
        Account a3 = new Account("AC1003", "Diya Shah", 89000.0);

        System.out.println("Saving accounts using varargs saveAll(T... items):");
        accountRepo.saveAll(a1, a2, a3);
        System.out.println("Total stored entities: " + accountRepo.count());

        System.out.println();
        System.out.println("Querying entity by ID via findById('AC1002'):");
        Account found = accountRepo.findById("AC1002");
        System.out.println("  Result: " + found);

        System.out.println();
        System.out.println("Listing all accounts via findAll():");
        for (Account a : accountRepo.findAll()) {
            System.out.println("  " + a);
        }

        System.out.println();
        System.out.println("Reusing the exact same generic Repository for Customer entities:");
        Repository<Customer> customerRepo = new Repository<>();
        customerRepo.save(new Customer("CUST101", "Rudra Modi", "rudra@charusat.edu.in"));
        customerRepo.save(new Customer("CUST102", "Kavya Patel", "kavya@gmail.com"));
```

```
        System.out.println("Customer count: " + customerRepo.count());
        System.out.println("Fetched Customer: " + customerRepo.findById("CUST101"));

        Double[] balances = { a1.getBalance(), a2.getBalance(), a3.getBalance() };
        System.out.println();
        System.out.printf("Average Account Balance (Bounded Generic Method): Rs. %.2f%n",
                Repository.average(balances));
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-13\PartB.java
RUDRA MODI || 25DCS058

Practical 13 - Part B: MiniBank Generic Repository Pattern with Reflection @Id

Saving accounts using varargs saveAll(T... items):
Total stored entities: 3

Querying entity by ID via findById('AC1002'):
  Result: Account[No=AC1002, Owner=Aarav Patel, Balance=Rs. 42000.00]

Listing all accounts via findAll():
  Account[No=AC1001, Owner=Rudra Modi, Balance=Rs. 75000.00]
  Account[No=AC1002, Owner=Aarav Patel, Balance=Rs. 42000.00]
  Account[No=AC1003, Owner=Diya Shah, Balance=Rs. 89000.00]

Reusing the exact same generic Repository for Customer entities:
Customer count: 2
Fetched Customer: Customer[ID=CUST101, Name=Rudra Modi, Email=rudra@charusat.edu.in]

Average Account Balance (Bounded Generic Method): Rs. 68666.67
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: How does the generic Repository<T> use reflection to dynamically discover the primary key?

Answer: The generic Repository<T> iterates through all declared fields of the passed entity class using entity.getClass().getDeclaredFields(). It checks for the presence of the @Id annotation using field.isAnnotationPresent(Id.class). Upon locating the annotated identifier field, it grants accessibility via field.setAccessible(true) and retrieves its value with field.get(entity).toString(), establishing a universal, reusable data access layer for any domain model.

Q2: Why is @SafeVarargs required on the saveAll(T... items) repository method?

Answer: In Java, varargs methods with generic component types create an underlying generic array (T[]). Because generic arrays are reifiable-unsafe due to type erasure, the compiler produces a heap pollution warning. The @SafeVarargs annotation asserts to the compiler that the method does not perform unsafe operations (such as storing an incompatible type into the varargs array or escaping the array reference), cleanly suppressing compiler warnings on final or static methods.